

BIPES Generated DSL Files

Combined export of generated block definitions, Python generators, and toolbox definitions.

static/page/blocks/blocks/clock_helpers_dsl.js

```
Blockly.Blocks["create_spi"] = {
  init: function() {
    this.appendDummyInput().appendField("Create Spi");
    this.appendValueInput("spi_id").setCheck("Number").appendField("Spi Id");
    this.appendValueInput("sck_pin").setCheck("Number").appendField("Sck Pin");
    this.appendValueInput("mosi_pin").setCheck("Number").appendField("Mosi Pin");
    this.appendValueInput("baudrate").setCheck("Number").appendField("Baudrate");
    this.appendValueInput("polarity").setCheck("Number").appendField("Polarity");
    this.appendValueInput("phase").setCheck("Number").appendField("Phase");
    this.setOutput(true, null);
    this.setColour(200);
    this.setInputsInline(false);
    this.setTooltip("Create an SPI object for display and device wrappers.");
    this.setHelpUrl("");
  }
};
```

static/page/blocks/blocks/dfplayer_dsl.js

```
Blockly.Blocks["dfplayer__create"] = {
  init: function() {
    this.appendDummyInput().appendField("Create DF Player");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("uart_id").setCheck("Number").appendField("UART ID");
    this.appendValueInput("tx_pin").setCheck("Number").appendField("TX Pin");
    this.appendValueInput("rx_pin").setCheck("Number").appendField("RX Pin");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(300);
    this.setInputsInline(false);
    this.setTooltip("Create a DFPlayer instance.");
    this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
  }
};

Blockly.Blocks["dfplayer__play"] = {
  init: function() {
    this.appendDummyInput().appendField("Play");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("track").setCheck("Number").appendField("Track");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(300);
    this.setInputsInline(false);
    this.setTooltip("");
    this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
  }
};

Blockly.Blocks["dfplayer__play_folder"] = {
  init: function() {
    this.appendDummyInput().appendField("Play Folder");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("folder").setCheck("Number").appendField("Folder");
    this.appendValueInput("track").setCheck("Number").appendField("Track");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(300);
    this.setInputsInline(false);
    this.setTooltip("");
  }
};
```

```

        this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
    }
};

Blockly.Blocks["dfplayer__play_mp3"] = {
    init: function() {
        this.appendDummyInput().appendField("Play Mp 3");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.appendValueInput("track").setCheck("Number").appendField("Track");
        this.appendValueInput("loop").setCheck("Boolean").appendField("Loop");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(300);
        this.setInputsInline(false);
        this.setTooltip("");
        this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
    }
};

Blockly.Blocks["dfplayer__pause"] = {
    init: function() {
        this.appendDummyInput().appendField("Pause");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(300);
        this.setInputsInline(false);
        this.setTooltip("");
        this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
    }
};

Blockly.Blocks["dfplayer__resume"] = {
    init: function() {
        this.appendDummyInput().appendField("Resume");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(300);
        this.setInputsInline(false);
        this.setTooltip("");
        this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
    }
};

Blockly.Blocks["dfplayer__stop"] = {
    init: function() {
        this.appendDummyInput().appendField("Stop");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(300);
        this.setInputsInline(false);
        this.setTooltip("");
        this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
    }
};

Blockly.Blocks["dfplayer__loop_current"] = {
    init: function() {
        this.appendDummyInput().appendField("Loop Current");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.appendValueInput("enable").setCheck("Boolean").appendField("Enable");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(300);
        this.setInputsInline(false);
        this.setTooltip("");
        this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
    }
};

```

```

Blockly.Blocks["dfplayer__next"] = {
  init: function() {
    this.appendDummyInput().appendField("Next");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(300);
    this.setInputsInline(false);
    this.setTooltip("");
    this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
  }
};

Blockly.Blocks["dfplayer__prev"] = {
  init: function() {
    this.appendDummyInput().appendField("Prev");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(300);
    this.setInputsInline(false);
    this.setTooltip("");
    this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
  }
};

Blockly.Blocks["dfplayer__volume"] = {
  init: function() {
    this.appendDummyInput().appendField("Volume");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("level").setCheck("Number").appendField("Level");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(300);
    this.setInputsInline(false);
    this.setTooltip("");
    this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
  }
};

Blockly.Blocks["dfplayer__volume_up"] = {
  init: function() {
    this.appendDummyInput().appendField("Volume Up");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(300);
    this.setInputsInline(false);
    this.setTooltip("");
    this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
  }
};

Blockly.Blocks["dfplayer__volume_down"] = {
  init: function() {
    this.appendDummyInput().appendField("Volume Down");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(300);
    this.setInputsInline(false);
    this.setTooltip("");
    this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
  }
};

Blockly.Blocks["dfplayer__eq"] = {
  init: function() {
    this.appendDummyInput().appendField("Eq");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendDummyInput().appendField("Mode").appendField(new Blockly.FieldDropdown([["Normal", "0"], ["Pop", "1"], ["Rock", "2"], ["Jazz", "3"], ["Classic", "4"], ["Bass", "5"]]), "mode");
  }
};

```

```

        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(300);
        this.setInputsInline(false);
        this.setTooltip("");
        this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
    }
};

Blockly.Blocks["dfplayer__reset"] = {
    init: function() {
        this.appendDummyInput().appendField("Reset");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(300);
        this.setInputsInline(false);
        this.setTooltip("");
        this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
    }
};

Blockly.Blocks["dfplayer__sleep"] = {
    init: function() {
        this.appendDummyInput().appendField("Sleep");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(300);
        this.setInputsInline(false);
        this.setTooltip("");
        this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
    }
};

Blockly.Blocks["dfplayer__wake"] = {
    init: function() {
        this.appendDummyInput().appendField("Wake");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(300);
        this.setInputsInline(false);
        this.setTooltip("");
        this.setHelpUrl("https://github.com/mannbro/PicoDFPlayer");
    }
};

```

static/page/blocks/blocks/ds1302_dsl.js

```

Blockly.Blocks["ds1302__create"] = {
    init: function() {
        this.appendDummyInput().appendField("Create DS1302");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.appendValueInput("clk_pin").setCheck("Number").appendField("CLK Pin");
        this.appendValueInput("dat_pin").setCheck("Number").appendField("DAT Pin");
        this.appendValueInput("rst_pin").setCheck("Number").appendField("RST Pin");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(35);
        this.setInputsInline(false);
        this.setTooltip("Create a DS1302 RTC instance.");
        this.setHelpUrl("https://github.com/micropython/micropython");
    }
};

Blockly.Blocks["ds1302__get_time"] = {
    init: function() {
        this.appendDummyInput().appendField("Get Time");
    }
};

```

```

        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setOutput(true, null);
        this.setColour(35);
        this.setInputsInline(false);
        this.setTooltip("Read the current time tuple from the RTC.");
        this.setHelpUrl("https://github.com/micropython/micropython");
    }
};

Blockly.Blocks["ds1302__get_year"] = {
    init: function() {
        this.appendDummyInput().appendField("Get Year");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setOutput(true, null);
        this.setColour(35);
        this.setInputsInline(false);
        this.setTooltip("Read the current year from the RTC.");
        this.setHelpUrl("https://github.com/micropython/micropython");
    }
};

Blockly.Blocks["ds1302__get_month"] = {
    init: function() {
        this.appendDummyInput().appendField("Get Month");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setOutput(true, null);
        this.setColour(35);
        this.setInputsInline(false);
        this.setTooltip("Read the current month from the RTC.");
        this.setHelpUrl("https://github.com/micropython/micropython");
    }
};

Blockly.Blocks["ds1302__get_day"] = {
    init: function() {
        this.appendDummyInput().appendField("Get Day");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setOutput(true, null);
        this.setColour(35);
        this.setInputsInline(false);
        this.setTooltip("Read the current day from the RTC.");
        this.setHelpUrl("https://github.com/micropython/micropython");
    }
};

Blockly.Blocks["ds1302__get_hour"] = {
    init: function() {
        this.appendDummyInput().appendField("Get Hour");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setOutput(true, null);
        this.setColour(35);
        this.setInputsInline(false);
        this.setTooltip("Read the current hour from the RTC.");
        this.setHelpUrl("https://github.com/micropython/micropython");
    }
};

Blockly.Blocks["ds1302__get_minute"] = {
    init: function() {
        this.appendDummyInput().appendField("Get Minute");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setOutput(true, null);
        this.setColour(35);
        this.setInputsInline(false);
        this.setTooltip("Read the current minute from the RTC.");
        this.setHelpUrl("https://github.com/micropython/micropython");
    }
};

Blockly.Blocks["ds1302__get_second"] = {
    init: function() {
        this.appendDummyInput().appendField("Get Second");
    }
};

```

```

        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setOutput(true, null);
        this.setColour(35);
        this.setInputsInline(false);
        this.setTooltip("Read the current second from the RTC.");
        this.setHelpUrl("https://github.com/micropython/micropython");
    }
};

Blockly.Blocks["ds1302__get_time_text"] = {
    init: function() {
        this.appendDummyInput().appendField("Get Time Text");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.appendValueInput("show_seconds").setCheck("Boolean").appendField("Show Seconds");
        this.setOutput(true, null);
        this.setColour(35);
        this.setInputsInline(false);
        this.setTooltip("Format the current time as text.");
        this.setHelpUrl("https://github.com/micropython/micropython");
    }
};

Blockly.Blocks["ds1302__get_date_text"] = {
    init: function() {
        this.appendDummyInput().appendField("Get Date Text");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setOutput(true, null);
        this.setColour(35);
        this.setInputsInline(false);
        this.setTooltip("Format the current date as text.");
        this.setHelpUrl("https://github.com/micropython/micropython");
    }
};

Blockly.Blocks["ds1302__set_time"] = {
    init: function() {
        this.appendDummyInput().appendField("Set Time");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.appendValueInput("year").setCheck("Number").appendField("Year");
        this.appendValueInput("month").setCheck("Number").appendField("Month");
        this.appendValueInput("day").setCheck("Number").appendField("Day");
        this.appendValueInput("hour").setCheck("Number").appendField("Hour");
        this.appendValueInput("minute").setCheck("Number").appendField("Minute");
        this.appendValueInput("second").setCheck("Number").appendField("Second");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(35);
        this.setInputsInline(false);
        this.setTooltip("Set the RTC date and time.");
        this.setHelpUrl("https://github.com/micropython/micropython");
    }
};

```

static/page/blocks/blocks/roboticsboard_dsl.js

```

Blockly.Blocks["kitronik_pico_robotics__create"] = {
    init: function() {
        this.appendDummyInput().appendField("Create Kitronik Pico Robotics");
        this.appendValueInput("I2CAddress").setCheck("Number").appendField("I2C Address");
        this.appendValueInput("sda").setCheck("Number").appendField("SDA");
        this.appendValueInput("scl").setCheck("Number").appendField("SCL");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(45);
        this.setInputsInline(false);
        this.setTooltip("Create and initialize the Kitronik Pico Robotics board.");
        this.setHelpUrl("https://github.com/KitronikLtd/Kitronik-Pico-Robotics-Board-MicroPython");
    }
};

```

```

Blockly.Blocks["kitronik_pico_robotics__adjust_servos"] = {
  init: function() {
    this.appendDummyInput().appendField("Adjust Servos");
    this.appendValueInput("change").setCheck("Number").appendField("Change");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(45);
    this.setInputsInline(false);
    this.setTooltip("Adjust the global servo calibration by a small amount.");
    this.setHelpUrl("https://github.com/KitronikLtd/Kitronik-Pico-Robotics-Board-MicroPython");
  }
};

Blockly.Blocks["kitronik_pico_robotics__servo_write"] = {
  init: function() {
    this.appendDummyInput().appendField("Servo Write");
    this.appendValueInput("servo").setCheck("Number").appendField("Servo");
    this.appendValueInput("degrees").setCheck("Number").appendField("Degrees");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(45);
    this.setInputsInline(false);
    this.setTooltip("Turn a servo motor to the requested angle in degrees.");
    this.setHelpUrl("https://github.com/KitronikLtd/Kitronik-Pico-Robotics-Board-MicroPython");
  }
};

Blockly.Blocks["kitronik_pico_robotics__servo_write_radians"] = {
  init: function() {
    this.appendDummyInput().appendField("Servo Write Radians");
    this.appendValueInput("servo").setCheck("Number").appendField("Servo");
    this.appendValueInput("radians").setCheck("Number").appendField("Radians");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(45);
    this.setInputsInline(false);
    this.setTooltip("Turn a servo motor to the requested angle in radians.");
    this.setHelpUrl("https://github.com/KitronikLtd/Kitronik-Pico-Robotics-Board-MicroPython");
  }
};

Blockly.Blocks["kitronik_pico_robotics__motor_on"] = {
  init: function() {
    this.appendDummyInput().appendField("Motor On");
    this.appendValueInput("motor").setCheck("Number").appendField("Motor");
    this.appendDummyInput().appendField("Direction").appendField(new Blockly.FieldDropdown([["Forward", "f"], ["Reverse", "r"]]), "direction");
    this.appendValueInput("speed").setCheck("Number").appendField("Speed");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(45);
    this.setInputsInline(false);
    this.setTooltip("Start a DC motor with direction and speed control.");
    this.setHelpUrl("https://github.com/KitronikLtd/Kitronik-Pico-Robotics-Board-MicroPython");
  }
};

Blockly.Blocks["kitronik_pico_robotics__motor_off"] = {
  init: function() {
    this.appendDummyInput().appendField("Motor Off");
    this.appendValueInput("motor").setCheck("Number").appendField("Motor");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(45);
    this.setInputsInline(false);
    this.setTooltip("Stop a DC motor.");
    this.setHelpUrl("https://github.com/KitronikLtd/Kitronik-Pico-Robotics-Board-MicroPython");
  }
};

Blockly.Blocks["kitronik_pico_robotics__step"] = {

```

```

init: function() {
  this.appendDummyInput().appendField("Step");
  this.appendValueInput("motor").setCheck("Number").appendField("Motor");
  this.appendDummyInput().appendField("Direction").appendField(new Blockly.FieldDropdown([["Forward", "f"], ["Reverse", "r"]]), "direction");
  this.appendValueInput("steps").setCheck("Number").appendField("Steps");
  this.appendValueInput("speed").setCheck("Number").appendField("Speed");
  this.appendValueInput("holdPosition").setCheck("Boolean").appendField("Hold Position");
  this.setPreviousStatement(true, null);
  this.setNextStatement(true, null);
  this.setColour(45);
  this.setInputsInline(false);
  this.setTooltip("Step a motor a fixed number of full steps.");
  this.setHelpUrl("https://github.com/KitronikLtd/Kitronik-Pico-Robotics-Board-MicroPython");
}
};

Blockly.Blocks["kitronik_pico_robotics__step_angle"] = {
  init: function() {
    this.appendDummyInput().appendField("Step Angle");
    this.appendValueInput("motor").setCheck("Number").appendField("Motor");
    this.appendDummyInput().appendField("Direction").appendField(new Blockly.FieldDropdown([["Forward", "f"], ["Reverse", "r"]]), "direction");
    this.appendValueInput("angle").setCheck("Number").appendField("Angle");
    this.appendValueInput("speed").setCheck("Number").appendField("Speed");
    this.appendValueInput("holdPosition").setCheck("Boolean").appendField("Hold Position");
    this.appendValueInput("stepsPerRev").setCheck("Number").appendField("Steps Per Rev");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(45);
    this.setInputsInline(false);
    this.setTooltip("Step a motor by a requested angle.");
    this.setHelpUrl("https://github.com/KitronikLtd/Kitronik-Pico-Robotics-Board-MicroPython");
  }
};

```

static/page/blocks/blocks/sand_table_robot_dsl.js

```

Blockly.Blocks["sand_table_robot__create"] = {
  init: function() {
    this.appendDummyInput().appendField("Create Sand Table Robot");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("motor1_pins").appendField("Motor 1 Pins");
    this.appendValueInput("motor2_pins").appendField("Motor 2 Pins");
    this.appendValueInput("sensor_shoulder_pin").setCheck("Number").appendField("Sensor Shoulder Pin");
    this.appendValueInput("sensor_elbow_pin").setCheck("Number").appendField("Sensor Elbow Pin");
    this.appendValueInput("L1").setCheck("Number").appendField("L1");
    this.appendValueInput("L2").setCheck("Number").appendField("L2");
    this.appendValueInput("steps_per_rev").setCheck("Number").appendField("Steps Per Rev");
    this.appendValueInput("backlash_deg_m1").setCheck("Number").appendField("Backlash Deg M 1");
    this.appendValueInput("backlash_deg_m2").setCheck("Number").appendField("Backlash Deg M 2");
    this.appendValueInput("homing_dir_shoulder").setCheck("Number").appendField("Homing Dir Shoulder");
    this.appendValueInput("homing_dir_elbow").setCheck("Number").appendField("Homing Dir Elbow");
    this.appendValueInput("homing_clear_steps").setCheck("Number").appendField("Homing Clear Steps");
    this.appendValueInput("default_speed_ms").setCheck("Number").appendField("Default Speed Ms");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(20);
    this.setInputsInline(false);
    this.setTooltip("Create a sand table robot instance.");
    this.setHelpUrl("");
  }
};

Blockly.Blocks["sand_table_robot__off"] = {
  init: function() {
    this.appendDummyInput().appendField("Off");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.setPreviousStatement(true, null);
  }
};

```



```

        this.setNextStatement(true, null);
        this.setColour(20);
        this.setInputsInline(false);
        this.setTooltip("Turn off both motors for a selected robot instance.");
        this.setHelpUrl("");
    }
};

Blockly.Blocks["sand_table_robot__home"] = {
    init: function() {
        this.appendDummyInput().appendField("Home");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(20);
        this.setInputsInline(false);
        this.setTooltip("Home the selected sand robot.");
        this.setHelpUrl("");
    }
};

Blockly.Blocks["sand_table_robot__move_line"] = {
    init: function() {
        this.appendDummyInput().appendField("Move Line");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.appendValueInput("target_x").setCheck("Number").appendField("Target X");
        this.appendValueInput("target_y").setCheck("Number").appendField("Target Y");
        this.appendValueInput("segments").setCheck("Number").appendField("Segments");
        this.appendValueInput("speed").setCheck("Number").appendField("Speed");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(20);
        this.setInputsInline(false);
        this.setTooltip("Move the robot in a straight line to the target point.");
        this.setHelpUrl("");
    }
};

Blockly.Blocks["sand_table_robot__move_arc"] = {
    init: function() {
        this.appendDummyInput().appendField("Move Arc");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.appendValueInput("center_x").setCheck("Number").appendField("Center X");
        this.appendValueInput("center_y").setCheck("Number").appendField("Center Y");
        this.appendValueInput("radius").setCheck("Number").appendField("Radius");
        this.appendValueInput("start_angle").setCheck("Number").appendField("Start Angle");
        this.appendValueInput("end_angle").setCheck("Number").appendField("End Angle");
        this.appendValueInput("segments").setCheck("Number").appendField("Segments");
        this.appendValueInput("speed").setCheck("Number").appendField("Speed");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(20);
        this.setInputsInline(false);
        this.setTooltip("Draw an arc on the sand table.");
        this.setHelpUrl("");
    }
};

Blockly.Blocks["sand_table_robot__draw_spiral"] = {
    init: function() {
        this.appendDummyInput().appendField("Draw Spiral");
        this.appendValueInput("id").setCheck("Number").appendField("ID #");
        this.appendValueInput("max_radius").setCheck("Number").appendField("Max Radius");
        this.appendValueInput("vindinger").setCheck("Number").appendField("Vindinger");
        this.appendValueInput("segments_pr_omgang").setCheck("Number").appendField("Segments Pr Omgang");
        this.appendValueInput("speed").setCheck("Number").appendField("Speed");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(20);
        this.setInputsInline(false);
        this.setTooltip("Draw a spiral pattern.");
        this.setHelpUrl("");
    }
};

```

```

    }
};

Blockly.Blocks["sand_table_robot__draw_flower"] = {
  init: function() {
    this.appendDummyInput().appendField("Draw Flower");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("max_radius").setCheck("Number").appendField("Max Radius");
    this.appendValueInput("petals").setCheck("Number").appendField("Petals");
    this.appendValueInput("speed").setCheck("Number").appendField("Speed");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(20);
    this.setInputsInline(false);
    this.setTooltip("Draw a flower pattern.");
    this.setHelpUrl("");
  }
};

Blockly.Blocks["sand_table_robot__run_gcode_text"] = {
  init: function() {
    this.appendDummyInput().appendField("Run Gcode Text");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("gcode_text").appendField("Gcode Text");
    this.appendValueInput("segments").setCheck("Number").appendField("Segments");
    this.appendValueInput("draw_speed_ms").setCheck("Number").appendField("Draw Speed Ms");
    this.appendValueInput("travel_speed_ms").setCheck("Number").appendField("Travel Speed Ms");
    this.appendValueInput("reset_modal").setCheck("Boolean").appendField("Reset Modal");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(20);
    this.setInputsInline(false);
    this.setTooltip("Run G-code from a text string.");
    this.setHelpUrl("");
  }
};

Blockly.Blocks["sand_table_robot__run_gcode_file"] = {
  init: function() {
    this.appendDummyInput().appendField("Run Gcode File");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("path").appendField("Path");
    this.appendValueInput("segments").setCheck("Number").appendField("Segments");
    this.appendValueInput("draw_speed_ms").setCheck("Number").appendField("Draw Speed Ms");
    this.appendValueInput("travel_speed_ms").setCheck("Number").appendField("Travel Speed Ms");
    this.appendValueInput("reset_modal").setCheck("Boolean").appendField("Reset Modal");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(20);
    this.setInputsInline(false);
    this.setTooltip("Run G-code from a file path.");
    this.setHelpUrl("");
  }
};

```

static/page/blocks/blocks/st7735s_dsl.js

```

Blockly.Blocks["color565"] = {
  init: function() {
    this.appendDummyInput().appendField("Color 565");
    this.appendValueInput("r").setCheck("Number").appendField("R");
    this.appendValueInput("g").setCheck("Number").appendField("G");
    this.appendValueInput("b").setCheck("Number").appendField("B");
    this.setOutput(true, null);
    this.setColour(210);
    this.setInputsInline(false);
    this.setTooltip("Convert RGB888 values into a 16-bit display color.");
    this.setHelpUrl("");
  }
};

```

```

};

Blockly.Blocks["st7735_s__create"] = {
  init: function() {
    this.appendDummyInput().appendField("Create ST7735S");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendDummyInput().appendField("SPI").appendField(new Blockly.FieldVariable("spi"), "spi");
    this.appendValueInput("dc_pin").setCheck("Number").appendField("DC Pin");
    this.appendValueInput("rst_pin").setCheck("Number").appendField("RST Pin");
    this.appendValueInput("cs_pin").setCheck("Number").appendField("CS Pin");
    this.appendValueInput("width").setCheck("Number").appendField("Width");
    this.appendValueInput("height").setCheck("Number").appendField("Height");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(210);
    this.setInputsInline(false);
    this.setTooltip("Create an ST7735S display instance.");
    this.setHelpUrl("https://github.com/micropython/micropython");
  }
};

Blockly.Blocks["st7735_s__fill"] = {
  init: function() {
    this.appendDummyInput().appendField("Fill");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("color").setCheck("Number").appendField("Color");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(210);
    this.setInputsInline(false);
    this.setTooltip("");
    this.setHelpUrl("https://github.com/micropython/micropython");
  }
};

Blockly.Blocks["st7735_s__fill_rect"] = {
  init: function() {
    this.appendDummyInput().appendField("Fill Rect");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("x").setCheck("Number").appendField("X");
    this.appendValueInput("y").setCheck("Number").appendField("Y");
    this.appendValueInput("w").setCheck("Number").appendField("W");
    this.appendValueInput("h").setCheck("Number").appendField("H");
    this.appendValueInput("color").setCheck("Number").appendField("Color");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(210);
    this.setInputsInline(false);
    this.setTooltip("");
    this.setHelpUrl("https://github.com/micropython/micropython");
  }
};

Blockly.Blocks["st7735_s__pixel"] = {
  init: function() {
    this.appendDummyInput().appendField("Pixel");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("x").setCheck("Number").appendField("X");
    this.appendValueInput("y").setCheck("Number").appendField("Y");
    this.appendValueInput("color").setCheck("Number").appendField("Color");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(210);
    this.setInputsInline(false);
    this.setTooltip("");
    this.setHelpUrl("https://github.com/micropython/micropython");
  }
};

Blockly.Blocks["st7735_s__hline"] = {
  init: function() {
    this.appendDummyInput().appendField("Hline");

```

```

    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("x").setCheck("Number").appendField("X");
    this.appendValueInput("y").setCheck("Number").appendField("Y");
    this.appendValueInput("w").setCheck("Number").appendField("W");
    this.appendValueInput("color").setCheck("Number").appendField("Color");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(210);
    this.setInputsInline(false);
    this.setTooltip("");
    this.setHelpUrl("https://github.com/micropython/micropython");
  }
};

```

```

Blockly.Blocks["st7735_s_vline"] = {
  init: function() {
    this.appendDummyInput().appendField("Vline");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("x").setCheck("Number").appendField("X");
    this.appendValueInput("y").setCheck("Number").appendField("Y");
    this.appendValueInput("h").setCheck("Number").appendField("H");
    this.appendValueInput("color").setCheck("Number").appendField("Color");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(210);
    this.setInputsInline(false);
    this.setTooltip("");
    this.setHelpUrl("https://github.com/micropython/micropython");
  }
};

```

```

Blockly.Blocks["st7735_s_rect"] = {
  init: function() {
    this.appendDummyInput().appendField("Rect");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("x").setCheck("Number").appendField("X");
    this.appendValueInput("y").setCheck("Number").appendField("Y");
    this.appendValueInput("w").setCheck("Number").appendField("W");
    this.appendValueInput("h").setCheck("Number").appendField("H");
    this.appendValueInput("color").setCheck("Number").appendField("Color");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(210);
    this.setInputsInline(false);
    this.setTooltip("");
    this.setHelpUrl("https://github.com/micropython/micropython");
  }
};

```

```

Blockly.Blocks["st7735_s_line"] = {
  init: function() {
    this.appendDummyInput().appendField("Line");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("x0").setCheck("Number").appendField("X 0");
    this.appendValueInput("y0").setCheck("Number").appendField("Y 0");
    this.appendValueInput("x1").setCheck("Number").appendField("X 1");
    this.appendValueInput("y1").setCheck("Number").appendField("Y 1");
    this.appendValueInput("color").setCheck("Number").appendField("Color");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(210);
    this.setInputsInline(false);
    this.setTooltip("");
    this.setHelpUrl("https://github.com/micropython/micropython");
  }
};

```

```

Blockly.Blocks["st7735_s_text"] = {
  init: function() {
    this.appendDummyInput().appendField("Text");
    this.appendValueInput("id").setCheck("Number").appendField("ID #");
    this.appendValueInput("string").appendField("String");
  }
};

```

```

    this.appendValueInput("x").setCheck("Number").appendField("X");
    this.appendValueInput("y").setCheck("Number").appendField("Y");
    this.appendValueInput("color").setCheck("Number").appendField("Color");
    this.appendValueInput("size").setCheck("Number").appendField("Size");
    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(210);
    this.setInputsInline(false);
    this.setTooltip("");
    this.setHelpUrl("https://github.com/micropython/micropython");
  }
};

```

static/page/blocks/pythonic/clock_helpers_dsl.js

```

Blockly.Python["create_spi"] = function(block) {
  Blockly.Python.definitions_["import_clock_helpers"] = "import clock_helpers";
  var spi_id = Blockly.Python.valueToCode(block, "spi_id", Blockly.Python.ORDER_ATOMIC);
  var sck_pin = Blockly.Python.valueToCode(block, "sck_pin", Blockly.Python.ORDER_ATOMIC);
  var mosi_pin = Blockly.Python.valueToCode(block, "mosi_pin", Blockly.Python.ORDER_ATOMIC);
  var baudrate = Blockly.Python.valueToCode(block, "baudrate", Blockly.Python.ORDER_ATOMIC);
  var polarity = Blockly.Python.valueToCode(block, "polarity", Blockly.Python.ORDER_ATOMIC);
  var phase = Blockly.Python.valueToCode(block, "phase", Blockly.Python.ORDER_ATOMIC);
  var code = "create_spi(" + spi_id + ", " + sck_pin + ", " + mosi_pin + ", " + baudrate + ", " + polarity + ", " + phase
+ ")" + "\n";
  return [code.trimEnd(), Blockly.Python.ORDER_NONE];
};

```

static/page/blocks/pythonic/dfplayer_dsl.js

```

Blockly.Python["dfplayer__create"] = function(block) {
  Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
  Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var uart_id = Blockly.Python.valueToCode(block, "uart_id", Blockly.Python.ORDER_ATOMIC);
  var tx_pin = Blockly.Python.valueToCode(block, "tx_pin", Blockly.Python.ORDER_ATOMIC);
  var rx_pin = Blockly.Python.valueToCode(block, "rx_pin", Blockly.Python.ORDER_ATOMIC);
  var code = "dfplayer_instances[" + id + "] = dfplayer.DFPlayer(" + id + ", " + uart_id + ", " + tx_pin + ", " + rx_pin +
+ ")" + "\n";
  return code;
};

Blockly.Python["dfplayer__play"] = function(block) {
  Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
  Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var track = Blockly.Python.valueToCode(block, "track", Blockly.Python.ORDER_ATOMIC);
  var code = "dfplayer_instances[" + id + "].play(" + track + ")" + "\n";
  return code;
};

Blockly.Python["dfplayer__play_folder"] = function(block) {
  Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
  Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var folder = Blockly.Python.valueToCode(block, "folder", Blockly.Python.ORDER_ATOMIC);
  var track = Blockly.Python.valueToCode(block, "track", Blockly.Python.ORDER_ATOMIC);
  var code = "dfplayer_instances[" + id + "].play_folder(" + folder + ", " + track + ")" + "\n";
  return code;
};

Blockly.Python["dfplayer__play_mp3"] = function(block) {
  Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
  Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var track = Blockly.Python.valueToCode(block, "track", Blockly.Python.ORDER_ATOMIC);
  var loop = Blockly.Python.valueToCode(block, "loop", Blockly.Python.ORDER_ATOMIC);

```

```

    var code = "dfplayer_instances[" + id + "].play_mp3(" + track + ", " + loop + ")" + "\n";
    return code;
};

Blockly.Python["dfplayer__pause"] = function(block) {
    Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
    Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
    var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
    var code = "dfplayer_instances[" + id + "].pause()" + "\n";
    return code;
};

Blockly.Python["dfplayer__resume"] = function(block) {
    Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
    Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
    var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
    var code = "dfplayer_instances[" + id + "].resume()" + "\n";
    return code;
};

Blockly.Python["dfplayer__stop"] = function(block) {
    Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
    Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
    var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
    var code = "dfplayer_instances[" + id + "].stop()" + "\n";
    return code;
};

Blockly.Python["dfplayer__loop_current"] = function(block) {
    Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
    Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
    var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
    var enable = Blockly.Python.valueToCode(block, "enable", Blockly.Python.ORDER_ATOMIC);
    var code = "dfplayer_instances[" + id + "].loop_current(" + enable + ")" + "\n";
    return code;
};

Blockly.Python["dfplayer__next"] = function(block) {
    Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
    Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
    var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
    var code = "dfplayer_instances[" + id + "].next()" + "\n";
    return code;
};

Blockly.Python["dfplayer__prev"] = function(block) {
    Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
    Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
    var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
    var code = "dfplayer_instances[" + id + "].prev()" + "\n";
    return code;
};

Blockly.Python["dfplayer__volume"] = function(block) {
    Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
    Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
    var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
    var level = Blockly.Python.valueToCode(block, "level", Blockly.Python.ORDER_ATOMIC);
    var code = "dfplayer_instances[" + id + "].volume(" + level + ")" + "\n";
    return code;
};

Blockly.Python["dfplayer__volume_up"] = function(block) {
    Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
    Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
    var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
    var code = "dfplayer_instances[" + id + "].volume_up()" + "\n";
    return code;
};

Blockly.Python["dfplayer__volume_down"] = function(block) {
    Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";

```

```

Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
var code = "dfplayer_instances[" + id + "].volume_down()" + "\n";
return code;
};

Blockly.Python["dfplayer__eq"] = function(block) {
  Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
  Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var mode = String(Number(block.getFieldValue("mode")));
  var code = "dfplayer_instances[" + id + "].eq(" + mode + ")" + "\n";
  return code;
};

Blockly.Python["dfplayer__reset"] = function(block) {
  Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
  Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var code = "dfplayer_instances[" + id + "].reset()" + "\n";
  return code;
};

Blockly.Python["dfplayer__sleep"] = function(block) {
  Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
  Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var code = "dfplayer_instances[" + id + "].sleep()" + "\n";
  return code;
};

Blockly.Python["dfplayer__wake"] = function(block) {
  Blockly.Python.definitions_["import_dfplayer"] = "import dfplayer";
  Blockly.Python.definitions_["registry_dfplayer_instances"] = "dfplayer_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var code = "dfplayer_instances[" + id + "].wake()" + "\n";
  return code;
};

```

static/page/blocks/pythonic/ds1302_dsl.js

```

Blockly.Python["ds1302__create"] = function(block) {
  Blockly.Python.definitions_["import_ds1302"] = "import ds1302";
  Blockly.Python.definitions_["registry_ds1302_instances"] = "ds1302_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var clk_pin = Blockly.Python.valueToCode(block, "clk_pin", Blockly.Python.ORDER_ATOMIC);
  var dat_pin = Blockly.Python.valueToCode(block, "dat_pin", Blockly.Python.ORDER_ATOMIC);
  var rst_pin = Blockly.Python.valueToCode(block, "rst_pin", Blockly.Python.ORDER_ATOMIC);
  var code = "ds1302_instances[" + id + "] = ds1302.DS1302(" + id + ", " + clk_pin + ", " + dat_pin + ", " + rst_pin + ")"
  + "\n";
  return code;
};

Blockly.Python["ds1302__get_time"] = function(block) {
  Blockly.Python.definitions_["import_ds1302"] = "import ds1302";
  Blockly.Python.definitions_["registry_ds1302_instances"] = "ds1302_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var code = "ds1302_instances[" + id + "].get_time()" + "\n";
  return [code.trimEnd(), Blockly.Python.ORDER_NONE];
};

Blockly.Python["ds1302__get_year"] = function(block) {
  Blockly.Python.definitions_["import_ds1302"] = "import ds1302";
  Blockly.Python.definitions_["registry_ds1302_instances"] = "ds1302_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var code = "ds1302_instances[" + id + "].get_year()" + "\n";
  return [code.trimEnd(), Blockly.Python.ORDER_NONE];
};

```

```

Blockly.Python["ds1302__get_month"] = function(block) {
  Blockly.Python.definitions_["import_ds1302"] = "import ds1302";
  Blockly.Python.definitions_["registry_ds1302_instances"] = "ds1302_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var code = "ds1302_instances[" + id + "].get_month()" + "\n";
  return [code.trimEnd(), Blockly.Python.ORDER_NONE];
};

Blockly.Python["ds1302__get_day"] = function(block) {
  Blockly.Python.definitions_["import_ds1302"] = "import ds1302";
  Blockly.Python.definitions_["registry_ds1302_instances"] = "ds1302_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var code = "ds1302_instances[" + id + "].get_day()" + "\n";
  return [code.trimEnd(), Blockly.Python.ORDER_NONE];
};

Blockly.Python["ds1302__get_hour"] = function(block) {
  Blockly.Python.definitions_["import_ds1302"] = "import ds1302";
  Blockly.Python.definitions_["registry_ds1302_instances"] = "ds1302_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var code = "ds1302_instances[" + id + "].get_hour()" + "\n";
  return [code.trimEnd(), Blockly.Python.ORDER_NONE];
};

Blockly.Python["ds1302__get_minute"] = function(block) {
  Blockly.Python.definitions_["import_ds1302"] = "import ds1302";
  Blockly.Python.definitions_["registry_ds1302_instances"] = "ds1302_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var code = "ds1302_instances[" + id + "].get_minute()" + "\n";
  return [code.trimEnd(), Blockly.Python.ORDER_NONE];
};

Blockly.Python["ds1302__get_second"] = function(block) {
  Blockly.Python.definitions_["import_ds1302"] = "import ds1302";
  Blockly.Python.definitions_["registry_ds1302_instances"] = "ds1302_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var code = "ds1302_instances[" + id + "].get_second()" + "\n";
  return [code.trimEnd(), Blockly.Python.ORDER_NONE];
};

Blockly.Python["ds1302__get_time_text"] = function(block) {
  Blockly.Python.definitions_["import_ds1302"] = "import ds1302";
  Blockly.Python.definitions_["registry_ds1302_instances"] = "ds1302_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var show_seconds = Blockly.Python.valueToCode(block, "show_seconds", Blockly.Python.ORDER_ATOMIC);
  var code = "ds1302_instances[" + id + "].get_time_text(" + show_seconds + ")" + "\n";
  return [code.trimEnd(), Blockly.Python.ORDER_NONE];
};

Blockly.Python["ds1302__get_date_text"] = function(block) {
  Blockly.Python.definitions_["import_ds1302"] = "import ds1302";
  Blockly.Python.definitions_["registry_ds1302_instances"] = "ds1302_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var code = "ds1302_instances[" + id + "].get_date_text()" + "\n";
  return [code.trimEnd(), Blockly.Python.ORDER_NONE];
};

Blockly.Python["ds1302__set_time"] = function(block) {
  Blockly.Python.definitions_["import_ds1302"] = "import ds1302";
  Blockly.Python.definitions_["registry_ds1302_instances"] = "ds1302_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var year = Blockly.Python.valueToCode(block, "year", Blockly.Python.ORDER_ATOMIC);
  var month = Blockly.Python.valueToCode(block, "month", Blockly.Python.ORDER_ATOMIC);
  var day = Blockly.Python.valueToCode(block, "day", Blockly.Python.ORDER_ATOMIC);
  var hour = Blockly.Python.valueToCode(block, "hour", Blockly.Python.ORDER_ATOMIC);
  var minute = Blockly.Python.valueToCode(block, "minute", Blockly.Python.ORDER_ATOMIC);
  var second = Blockly.Python.valueToCode(block, "second", Blockly.Python.ORDER_ATOMIC);
  var code = "ds1302_instances[" + id + "].set_time(" + year + ", " + month + ", " + day + ", " + hour + ", " + minute +
", " + second + ")" + "\n";
  return code;
};

```



```
Blockly.Python["kitronik_pico_robotics__create"] = function(block) {
  Blockly.Python.definitions_["import_PicoRobotics"] = "import PicoRobotics";
  var I2CAddress = Blockly.Python.valueToCode(block, "I2CAddress", Blockly.Python.ORDER_ATOMIC);
  var sda = Blockly.Python.valueToCode(block, "sda", Blockly.Python.ORDER_ATOMIC);
  var scl = Blockly.Python.valueToCode(block, "scl", Blockly.Python.ORDER_ATOMIC);
  var code = "board = PicoRobotics.KitronikPicoRobotics(" + I2CAddress + ", " + sda + ", " + scl + ")" + "\n";
  return code;
};

Blockly.Python["kitronik_pico_robotics__adjust_servos"] = function(block) {
  Blockly.Python.definitions_["import_PicoRobotics"] = "import PicoRobotics";
  var change = Blockly.Python.valueToCode(block, "change", Blockly.Python.ORDER_ATOMIC);
  var code = "board.adjustServos(" + change + ")" + "\n";
  return code;
};

Blockly.Python["kitronik_pico_robotics__servo_write"] = function(block) {
  Blockly.Python.definitions_["import_PicoRobotics"] = "import PicoRobotics";
  var servo = Blockly.Python.valueToCode(block, "servo", Blockly.Python.ORDER_ATOMIC);
  var degrees = Blockly.Python.valueToCode(block, "degrees", Blockly.Python.ORDER_ATOMIC);
  var code = "board.servoWrite(" + servo + ", " + degrees + ")" + "\n";
  return code;
};

Blockly.Python["kitronik_pico_robotics__servo_write_radians"] = function(block) {
  Blockly.Python.definitions_["import_PicoRobotics"] = "import PicoRobotics";
  var servo = Blockly.Python.valueToCode(block, "servo", Blockly.Python.ORDER_ATOMIC);
  var radians = Blockly.Python.valueToCode(block, "radians", Blockly.Python.ORDER_ATOMIC);
  var code = "board.servoWriteRadians(" + servo + ", " + radians + ")" + "\n";
  return code;
};

Blockly.Python["kitronik_pico_robotics__motor_on"] = function(block) {
  Blockly.Python.definitions_["import_PicoRobotics"] = "import PicoRobotics";
  var motor = Blockly.Python.valueToCode(block, "motor", Blockly.Python.ORDER_ATOMIC);
  var direction = JSON.stringify(block.getFieldValue("direction"));
  var speed = Blockly.Python.valueToCode(block, "speed", Blockly.Python.ORDER_ATOMIC);
  var code = "board.motorOn(" + motor + ", " + direction + ", " + speed + ")" + "\n";
  return code;
};

Blockly.Python["kitronik_pico_robotics__motor_off"] = function(block) {
  Blockly.Python.definitions_["import_PicoRobotics"] = "import PicoRobotics";
  var motor = Blockly.Python.valueToCode(block, "motor", Blockly.Python.ORDER_ATOMIC);
  var code = "board.motorOff(" + motor + ")" + "\n";
  return code;
};

Blockly.Python["kitronik_pico_robotics__step"] = function(block) {
  Blockly.Python.definitions_["import_PicoRobotics"] = "import PicoRobotics";
  var motor = Blockly.Python.valueToCode(block, "motor", Blockly.Python.ORDER_ATOMIC);
  var direction = JSON.stringify(block.getFieldValue("direction"));
  var steps = Blockly.Python.valueToCode(block, "steps", Blockly.Python.ORDER_ATOMIC);
  var speed = Blockly.Python.valueToCode(block, "speed", Blockly.Python.ORDER_ATOMIC);
  var holdPosition = Blockly.Python.valueToCode(block, "holdPosition", Blockly.Python.ORDER_ATOMIC);
  var code = "board.step(" + motor + ", " + direction + ", " + steps + ", " + speed + ", " + holdPosition + ")" + "\n";
  return code;
};

Blockly.Python["kitronik_pico_robotics__step_angle"] = function(block) {
  Blockly.Python.definitions_["import_PicoRobotics"] = "import PicoRobotics";
  var motor = Blockly.Python.valueToCode(block, "motor", Blockly.Python.ORDER_ATOMIC);
  var direction = JSON.stringify(block.getFieldValue("direction"));
  var angle = Blockly.Python.valueToCode(block, "angle", Blockly.Python.ORDER_ATOMIC);
  var speed = Blockly.Python.valueToCode(block, "speed", Blockly.Python.ORDER_ATOMIC);
  var holdPosition = Blockly.Python.valueToCode(block, "holdPosition", Blockly.Python.ORDER_ATOMIC);
  var stepsPerRev = Blockly.Python.valueToCode(block, "stepsPerRev", Blockly.Python.ORDER_ATOMIC);
  var code = "board.stepAngle(" + motor + ", " + direction + ", " + angle + ", " + speed + ", " + holdPosition + ", " + stepsPerRev + ")" + "\n";
  return code;
};
```

```
};
```

static/page/blocks/pythonic/sand_table_robot_dsl.js

```
Blockly.Python["sand_table_robot__create"] = function(block) {
  Blockly.Python.definitions_["import_sand_table_robot"] = "import sand_table_robot";
  Blockly.Python.definitions_["registry_sand_table_robot_instances"] = "sand_table_robot_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var motor1_pins = Blockly.Python.valueToCode(block, "motor1_pins", Blockly.Python.ORDER_ATOMIC);
  var motor2_pins = Blockly.Python.valueToCode(block, "motor2_pins", Blockly.Python.ORDER_ATOMIC);
  var sensor_shoulder_pin = Blockly.Python.valueToCode(block, "sensor_shoulder_pin", Blockly.Python.ORDER_ATOMIC);
  var sensor_elbow_pin = Blockly.Python.valueToCode(block, "sensor_elbow_pin", Blockly.Python.ORDER_ATOMIC);
  var L1 = Blockly.Python.valueToCode(block, "L1", Blockly.Python.ORDER_ATOMIC);
  var L2 = Blockly.Python.valueToCode(block, "L2", Blockly.Python.ORDER_ATOMIC);
  var steps_per_rev = Blockly.Python.valueToCode(block, "steps_per_rev", Blockly.Python.ORDER_ATOMIC);
  var backlash_deg_m1 = Blockly.Python.valueToCode(block, "backlash_deg_m1", Blockly.Python.ORDER_ATOMIC);
  var backlash_deg_m2 = Blockly.Python.valueToCode(block, "backlash_deg_m2", Blockly.Python.ORDER_ATOMIC);
  var homing_dir_shoulder = Blockly.Python.valueToCode(block, "homing_dir_shoulder", Blockly.Python.ORDER_ATOMIC);
  var homing_dir_elbow = Blockly.Python.valueToCode(block, "homing_dir_elbow", Blockly.Python.ORDER_ATOMIC);
  var homing_clear_steps = Blockly.Python.valueToCode(block, "homing_clear_steps", Blockly.Python.ORDER_ATOMIC);
  var default_speed_ms = Blockly.Python.valueToCode(block, "default_speed_ms", Blockly.Python.ORDER_ATOMIC);
  var code = "sand_table_robot_instances[" + id + "] = sand_table_robot.SandTableRobot(" + id + ", " + motor1_pins + ", " +
+ motor2_pins + ", " + sensor_shoulder_pin + ", " + sensor_elbow_pin + ", " + L1 + ", " + L2 + ", " + steps_per_rev + ", " +
+ backlash_deg_m1 + ", " + backlash_deg_m2 + ", " + homing_dir_shoulder + ", " + homing_dir_elbow + ", " +
+ homing_clear_steps + ", " + default_speed_ms + ")";
  return code;
};

Blockly.Python["sand_table_robot__off"] = function(block) {
  Blockly.Python.definitions_["import_sand_table_robot"] = "import sand_table_robot";
  Blockly.Python.definitions_["registry_sand_table_robot_instances"] = "sand_table_robot_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var code = "sand_table_robot_instances[" + id + "].off()" + "\n";
  return code;
};

Blockly.Python["sand_table_robot__home"] = function(block) {
  Blockly.Python.definitions_["import_sand_table_robot"] = "import sand_table_robot";
  Blockly.Python.definitions_["registry_sand_table_robot_instances"] = "sand_table_robot_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var code = "sand_table_robot_instances[" + id + "].home()" + "\n";
  return code;
};

Blockly.Python["sand_table_robot__move_line"] = function(block) {
  Blockly.Python.definitions_["import_sand_table_robot"] = "import sand_table_robot";
  Blockly.Python.definitions_["registry_sand_table_robot_instances"] = "sand_table_robot_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var target_x = Blockly.Python.valueToCode(block, "target_x", Blockly.Python.ORDER_ATOMIC);
  var target_y = Blockly.Python.valueToCode(block, "target_y", Blockly.Python.ORDER_ATOMIC);
  var segments = Blockly.Python.valueToCode(block, "segments", Blockly.Python.ORDER_ATOMIC);
  var speed = Blockly.Python.valueToCode(block, "speed", Blockly.Python.ORDER_ATOMIC);
  var code = "sand_table_robot_instances[" + id + "].move_line(" + target_x + ", " + target_y + ", " + segments + ", " +
+ speed + ")";
  return code;
};

Blockly.Python["sand_table_robot__move_arc"] = function(block) {
  Blockly.Python.definitions_["import_sand_table_robot"] = "import sand_table_robot";
  Blockly.Python.definitions_["registry_sand_table_robot_instances"] = "sand_table_robot_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var center_x = Blockly.Python.valueToCode(block, "center_x", Blockly.Python.ORDER_ATOMIC);
  var center_y = Blockly.Python.valueToCode(block, "center_y", Blockly.Python.ORDER_ATOMIC);
  var radius = Blockly.Python.valueToCode(block, "radius", Blockly.Python.ORDER_ATOMIC);
  var start_angle = Blockly.Python.valueToCode(block, "start_angle", Blockly.Python.ORDER_ATOMIC);
  var end_angle = Blockly.Python.valueToCode(block, "end_angle", Blockly.Python.ORDER_ATOMIC);
  var segments = Blockly.Python.valueToCode(block, "segments", Blockly.Python.ORDER_ATOMIC);
  var speed = Blockly.Python.valueToCode(block, "speed", Blockly.Python.ORDER_ATOMIC);
  var code = "sand_table_robot_instances[" + id + "].move_arc(" + center_x + ", " + center_y + ", " + radius + ", " +
```

```

start_angle + ", " + end_angle + ", " + segments + ", " + speed + ")" + "\n";
    return code;
};

Blockly.Python["sand_table_robot__draw_spiral"] = function(block) {
    Blockly.Python.definitions_["import_sand_table_robot"] = "import sand_table_robot";
    Blockly.Python.definitions_["registry_sand_table_robot_instances"] = "sand_table_robot_instances = {}";
    var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
    var max_radius = Blockly.Python.valueToCode(block, "max_radius", Blockly.Python.ORDER_ATOMIC);
    var vindinger = Blockly.Python.valueToCode(block, "vindinger", Blockly.Python.ORDER_ATOMIC);
    var segments_pr_omgang = Blockly.Python.valueToCode(block, "segments_pr_omgang", Blockly.Python.ORDER_ATOMIC);
    var speed = Blockly.Python.valueToCode(block, "speed", Blockly.Python.ORDER_ATOMIC);
    var code = "sand_table_robot_instances[" + id + "].draw_spiral(" + max_radius + ", " + vindinger + ", " +
segments_pr_omgang + ", " + speed + ")" + "\n";
    return code;
};

Blockly.Python["sand_table_robot__draw_flower"] = function(block) {
    Blockly.Python.definitions_["import_sand_table_robot"] = "import sand_table_robot";
    Blockly.Python.definitions_["registry_sand_table_robot_instances"] = "sand_table_robot_instances = {}";
    var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
    var max_radius = Blockly.Python.valueToCode(block, "max_radius", Blockly.Python.ORDER_ATOMIC);
    var petals = Blockly.Python.valueToCode(block, "petals", Blockly.Python.ORDER_ATOMIC);
    var speed = Blockly.Python.valueToCode(block, "speed", Blockly.Python.ORDER_ATOMIC);
    var code = "sand_table_robot_instances[" + id + "].draw_flower(" + max_radius + ", " + petals + ", " + speed + ")" + "\n";
    return code;
};

Blockly.Python["sand_table_robot__run_gcode_text"] = function(block) {
    Blockly.Python.definitions_["import_sand_table_robot"] = "import sand_table_robot";
    Blockly.Python.definitions_["registry_sand_table_robot_instances"] = "sand_table_robot_instances = {}";
    var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
    var gcode_text = Blockly.Python.valueToCode(block, "gcode_text", Blockly.Python.ORDER_ATOMIC);
    var segments = Blockly.Python.valueToCode(block, "segments", Blockly.Python.ORDER_ATOMIC);
    var draw_speed_ms = Blockly.Python.valueToCode(block, "draw_speed_ms", Blockly.Python.ORDER_ATOMIC);
    var travel_speed_ms = Blockly.Python.valueToCode(block, "travel_speed_ms", Blockly.Python.ORDER_ATOMIC);
    var reset_modal = Blockly.Python.valueToCode(block, "reset_modal", Blockly.Python.ORDER_ATOMIC);
    var code = "sand_table_robot_instances[" + id + "].run_gcode_text(" + gcode_text + ", " + segments + ", " +
draw_speed_ms + ", " + travel_speed_ms + ", " + reset_modal + ")" + "\n";
    return code;
};

Blockly.Python["sand_table_robot__run_gcode_file"] = function(block) {
    Blockly.Python.definitions_["import_sand_table_robot"] = "import sand_table_robot";
    Blockly.Python.definitions_["registry_sand_table_robot_instances"] = "sand_table_robot_instances = {}";
    var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
    var path = Blockly.Python.valueToCode(block, "path", Blockly.Python.ORDER_ATOMIC);
    var segments = Blockly.Python.valueToCode(block, "segments", Blockly.Python.ORDER_ATOMIC);
    var draw_speed_ms = Blockly.Python.valueToCode(block, "draw_speed_ms", Blockly.Python.ORDER_ATOMIC);
    var travel_speed_ms = Blockly.Python.valueToCode(block, "travel_speed_ms", Blockly.Python.ORDER_ATOMIC);
    var reset_modal = Blockly.Python.valueToCode(block, "reset_modal", Blockly.Python.ORDER_ATOMIC);
    var code = "sand_table_robot_instances[" + id + "].run_gcode_file(" + path + ", " + segments + ", " + draw_speed_ms + ",
" + travel_speed_ms + ", " + reset_modal + ")" + "\n";
    return code;
};

```

static/page/blocks/pythonic/st7735s_dsl.js

```

Blockly.Python["color565"] = function(block) {
    Blockly.Python.definitions_["import_st7735s"] = "import st7735s";
    var r = Blockly.Python.valueToCode(block, "r", Blockly.Python.ORDER_ATOMIC);
    var g = Blockly.Python.valueToCode(block, "g", Blockly.Python.ORDER_ATOMIC);
    var b = Blockly.Python.valueToCode(block, "b", Blockly.Python.ORDER_ATOMIC);
    var code = "color565(" + r + ", " + g + ", " + b + ")" + "\n";
    return [code.trimEnd(), Blockly.Python.ORDER_NONE];
};

Blockly.Python["st7735_s__create"] = function(block) {

```

```

Blockly.Python.definitions_["import_st7735s"] = "import st7735s";
Blockly.Python.definitions_["registry_st7735_s_instances"] = "st7735_s_instances = {}";
var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
$ = Blockly.internal_;
var spi = $.Blockly.Python.namedDB_.getName(block.getFieldValue("spi"),
$.module$exports$Blockly$Names.NameType.VARIABLE);
var dc_pin = Blockly.Python.valueToCode(block, "dc_pin", Blockly.Python.ORDER_ATOMIC);
var rst_pin = Blockly.Python.valueToCode(block, "rst_pin", Blockly.Python.ORDER_ATOMIC);
var cs_pin = Blockly.Python.valueToCode(block, "cs_pin", Blockly.Python.ORDER_ATOMIC);
var width = Blockly.Python.valueToCode(block, "width", Blockly.Python.ORDER_ATOMIC);
var height = Blockly.Python.valueToCode(block, "height", Blockly.Python.ORDER_ATOMIC);
var code = "st7735_s_instances[" + id + "] = st7735s.ST7735S(" + id + ", " + spi + ", " + dc_pin + ", " + rst_pin + ", "
+ cs_pin + ", " + width + ", " + height + ")" + "\n";
return code;
};

Blockly.Python["st7735_s__fill"] = function(block) {
  Blockly.Python.definitions_["import_st7735s"] = "import st7735s";
  Blockly.Python.definitions_["registry_st7735_s_instances"] = "st7735_s_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var color = Blockly.Python.valueToCode(block, "color", Blockly.Python.ORDER_ATOMIC);
  var code = "st7735_s_instances[" + id + "].fill(" + color + ")" + "\n";
  return code;
};

Blockly.Python["st7735_s__fill_rect"] = function(block) {
  Blockly.Python.definitions_["import_st7735s"] = "import st7735s";
  Blockly.Python.definitions_["registry_st7735_s_instances"] = "st7735_s_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var x = Blockly.Python.valueToCode(block, "x", Blockly.Python.ORDER_ATOMIC);
  var y = Blockly.Python.valueToCode(block, "y", Blockly.Python.ORDER_ATOMIC);
  var w = Blockly.Python.valueToCode(block, "w", Blockly.Python.ORDER_ATOMIC);
  var h = Blockly.Python.valueToCode(block, "h", Blockly.Python.ORDER_ATOMIC);
  var color = Blockly.Python.valueToCode(block, "color", Blockly.Python.ORDER_ATOMIC);
  var code = "st7735_s_instances[" + id + "].fill_rect(" + x + ", " + y + ", " + w + ", " + h + ", " + color + ")" + "\n";
  return code;
};

Blockly.Python["st7735_s__pixel"] = function(block) {
  Blockly.Python.definitions_["import_st7735s"] = "import st7735s";
  Blockly.Python.definitions_["registry_st7735_s_instances"] = "st7735_s_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var x = Blockly.Python.valueToCode(block, "x", Blockly.Python.ORDER_ATOMIC);
  var y = Blockly.Python.valueToCode(block, "y", Blockly.Python.ORDER_ATOMIC);
  var color = Blockly.Python.valueToCode(block, "color", Blockly.Python.ORDER_ATOMIC);
  var code = "st7735_s_instances[" + id + "].pixel(" + x + ", " + y + ", " + color + ")" + "\n";
  return code;
};

Blockly.Python["st7735_s__hline"] = function(block) {
  Blockly.Python.definitions_["import_st7735s"] = "import st7735s";
  Blockly.Python.definitions_["registry_st7735_s_instances"] = "st7735_s_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var x = Blockly.Python.valueToCode(block, "x", Blockly.Python.ORDER_ATOMIC);
  var y = Blockly.Python.valueToCode(block, "y", Blockly.Python.ORDER_ATOMIC);
  var w = Blockly.Python.valueToCode(block, "w", Blockly.Python.ORDER_ATOMIC);
  var color = Blockly.Python.valueToCode(block, "color", Blockly.Python.ORDER_ATOMIC);
  var code = "st7735_s_instances[" + id + "].hline(" + x + ", " + y + ", " + w + ", " + color + ")" + "\n";
  return code;
};

Blockly.Python["st7735_s__vline"] = function(block) {
  Blockly.Python.definitions_["import_st7735s"] = "import st7735s";
  Blockly.Python.definitions_["registry_st7735_s_instances"] = "st7735_s_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var x = Blockly.Python.valueToCode(block, "x", Blockly.Python.ORDER_ATOMIC);
  var y = Blockly.Python.valueToCode(block, "y", Blockly.Python.ORDER_ATOMIC);
  var h = Blockly.Python.valueToCode(block, "h", Blockly.Python.ORDER_ATOMIC);
  var color = Blockly.Python.valueToCode(block, "color", Blockly.Python.ORDER_ATOMIC);
  var code = "st7735_s_instances[" + id + "].vline(" + x + ", " + y + ", " + h + ", " + color + ")" + "\n";
  return code;
};

```

```

Blockly.Python["st7735_s__rect"] = function(block) {
  Blockly.Python.definitions_["import_st7735s"] = "import st7735s";
  Blockly.Python.definitions_["registry_st7735_s_instances"] = "st7735_s_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var x = Blockly.Python.valueToCode(block, "x", Blockly.Python.ORDER_ATOMIC);
  var y = Blockly.Python.valueToCode(block, "y", Blockly.Python.ORDER_ATOMIC);
  var w = Blockly.Python.valueToCode(block, "w", Blockly.Python.ORDER_ATOMIC);
  var h = Blockly.Python.valueToCode(block, "h", Blockly.Python.ORDER_ATOMIC);
  var color = Blockly.Python.valueToCode(block, "color", Blockly.Python.ORDER_ATOMIC);
  var code = "st7735_s_instances[" + id + "].rect(" + x + ", " + y + ", " + w + ", " + h + ", " + color + ")" + "\n";
  return code;
};

Blockly.Python["st7735_s__line"] = function(block) {
  Blockly.Python.definitions_["import_st7735s"] = "import st7735s";
  Blockly.Python.definitions_["registry_st7735_s_instances"] = "st7735_s_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var x0 = Blockly.Python.valueToCode(block, "x0", Blockly.Python.ORDER_ATOMIC);
  var y0 = Blockly.Python.valueToCode(block, "y0", Blockly.Python.ORDER_ATOMIC);
  var x1 = Blockly.Python.valueToCode(block, "x1", Blockly.Python.ORDER_ATOMIC);
  var y1 = Blockly.Python.valueToCode(block, "y1", Blockly.Python.ORDER_ATOMIC);
  var color = Blockly.Python.valueToCode(block, "color", Blockly.Python.ORDER_ATOMIC);
  var code = "st7735_s_instances[" + id + "].line(" + x0 + ", " + y0 + ", " + x1 + ", " + y1 + ", " + color + ")" + "\n";
  return code;
};

Blockly.Python["st7735_s__text"] = function(block) {
  Blockly.Python.definitions_["import_st7735s"] = "import st7735s";
  Blockly.Python.definitions_["registry_st7735_s_instances"] = "st7735_s_instances = {}";
  var id = Blockly.Python.valueToCode(block, "id", Blockly.Python.ORDER_ATOMIC);
  var string = Blockly.Python.valueToCode(block, "string", Blockly.Python.ORDER_ATOMIC);
  var x = Blockly.Python.valueToCode(block, "x", Blockly.Python.ORDER_ATOMIC);
  var y = Blockly.Python.valueToCode(block, "y", Blockly.Python.ORDER_ATOMIC);
  var color = Blockly.Python.valueToCode(block, "color", Blockly.Python.ORDER_ATOMIC);
  var size = Blockly.Python.valueToCode(block, "size", Blockly.Python.ORDER_ATOMIC);
  var code = "st7735_s_instances[" + id + "].text(" + string + ", " + x + ", " + y + ", " + color + ", " + size + ")" + "\n";
  return code;
};

```

templates/page/blocks/definitions/clock_helpers_dsl.md

```

# Clock Helpers
<category name="Clock Helpers">
<label text="Clock Helpers"></label>
<button text="%{INSTALL_LIBRARY}: clock_helpers" callbackKey="installPyLib"></button>

# create_spi
<block type="create_spi">
  <value name="spi_id">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="sck_pin">
    <shadow type="pinout">
      <field name="PIN">18</field>
    </shadow>
  </value>
  <value name="mosi_pin">
    <shadow type="pinout">
      <field name="PIN">19</field>
    </shadow>
  </value>
  <value name="baudrate">
    <shadow type="math_number">
      <field name="NUM">4000000</field>
    </shadow>

```

```

</value>
<value name="polarity">
  <shadow type="math_number">
    <field name="NUM">0</field>
  </shadow>
</value>
<value name="phase">
  <shadow type="math_number">
    <field name="NUM">0</field>
  </shadow>
</value>
</block>

```

templates/page/blocks/definitions/dfplayer_dsl.md

```

# DFPlayer
<category name="DFPlayer">
<label text="DFPlayer"></label>
<button text="%{INSTALL_LIBRARY}: dfplayer" callbackKey="installPyLib"></button>

# dfplayer__create
<block type="dfplayer__create">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="uart_id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="tx_pin">
    <shadow type="pinout">
      <field name="PIN">4</field>
    </shadow>
  </value>
  <value name="rx_pin">
    <shadow type="pinout">
      <field name="PIN">5</field>
    </shadow>
  </value>
</block>

# dfplayer__play
<block type="dfplayer__play">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="track">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

# dfplayer__play_folder
<block type="dfplayer__play_folder">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="folder">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>

```

```
</value>
<value name="track">
  <shadow type="math_number">
    <field name="NUM">0</field>
  </shadow>
</value>
</block>

# dfplayer__play_mp3
<block type="dfplayer__play_mp3">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="track">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="loop">
    <shadow type="logic_boolean">
      <field name="BOOL">FALSE</field>
    </shadow>
  </value>
</block>

# dfplayer__pause
<block type="dfplayer__pause">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

# dfplayer__resume
<block type="dfplayer__resume">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

# dfplayer__stop
<block type="dfplayer__stop">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

# dfplayer__loop_current
<block type="dfplayer__loop_current">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="enable">
    <shadow type="logic_boolean">
      <field name="BOOL">TRUE</field>
    </shadow>
  </value>
</block>

# dfplayer__next
<block type="dfplayer__next">
  <value name="id">
    <shadow type="math_number">
```

```
        <field name="NUM">1</field>
    </shadow>
</value>
</block>

# dfplayer__prev
<block type="dfplayer__prev">
    <value name="id">
        <shadow type="math_number">
            <field name="NUM">1</field>
        </shadow>
    </value>
</block>

# dfplayer__volume
<block type="dfplayer__volume">
    <value name="id">
        <shadow type="math_number">
            <field name="NUM">1</field>
        </shadow>
    </value>
    <value name="level">
        <shadow type="math_number">
            <field name="NUM">0</field>
        </shadow>
    </value>
</block>

# dfplayer__volume_up
<block type="dfplayer__volume_up">
    <value name="id">
        <shadow type="math_number">
            <field name="NUM">1</field>
        </shadow>
    </value>
</block>

# dfplayer__volume_down
<block type="dfplayer__volume_down">
    <value name="id">
        <shadow type="math_number">
            <field name="NUM">1</field>
        </shadow>
    </value>
</block>

# dfplayer__eq
<block type="dfplayer__eq">
    <value name="id">
        <shadow type="math_number">
            <field name="NUM">1</field>
        </shadow>
    </value>
</block>

# dfplayer__reset
<block type="dfplayer__reset">
    <value name="id">
        <shadow type="math_number">
            <field name="NUM">1</field>
        </shadow>
    </value>
</block>

# dfplayer__sleep
<block type="dfplayer__sleep">
    <value name="id">
        <shadow type="math_number">
            <field name="NUM">1</field>
        </shadow>
    </value>
</block>
```



```

# dfplayer__wake
<block type="dfplayer__wake">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

```

templates/page/blocks/definitions/ds1302_dsl.md

```

# DS1302
<category name="DS1302">
<label text="DS1302"></label>
<button text="%{INSTALL_LIBRARY}: ds1302" callbackKey="installPyLib"></button>

# ds1302__create
<block type="ds1302__create">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="clk_pin">
    <shadow type="pinout">
      <field name="PIN">0</field>
    </shadow>
  </value>
  <value name="dat_pin">
    <shadow type="pinout">
      <field name="PIN">0</field>
    </shadow>
  </value>
  <value name="rst_pin">
    <shadow type="pinout">
      <field name="PIN">0</field>
    </shadow>
  </value>
</block>

# ds1302__get_time
<block type="ds1302__get_time">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

# ds1302__get_year
<block type="ds1302__get_year">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

# ds1302__get_month
<block type="ds1302__get_month">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

# ds1302__get_day

```

```
<block type="ds1302__get_day">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

# ds1302__get_hour
<block type="ds1302__get_hour">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

# ds1302__get_minute
<block type="ds1302__get_minute">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

# ds1302__get_second
<block type="ds1302__get_second">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

# ds1302__get_time_text
<block type="ds1302__get_time_text">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="show_seconds">
    <shadow type="logic_boolean">
      <field name="BOOL">TRUE</field>
    </shadow>
  </value>
</block>

# ds1302__get_date_text
<block type="ds1302__get_date_text">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

# ds1302__set_time
<block type="ds1302__set_time">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="year">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="month">
    <shadow type="math_number">
```

```

        <field name="NUM">0</field>
    </shadow>
</value>
<value name="day">
    <shadow type="math_number">
        <field name="NUM">0</field>
    </shadow>
</value>
<value name="hour">
    <shadow type="math_number">
        <field name="NUM">0</field>
    </shadow>
</value>
<value name="minute">
    <shadow type="math_number">
        <field name="NUM">0</field>
    </shadow>
</value>
<value name="second">
    <shadow type="math_number">
        <field name="NUM">0</field>
    </shadow>
</value>
</block>

```

templates/page/blocks/definitions/roboticsboard_dsl.md

```

# Robotics Board
<category name="Robotics Board">
<label text="Robotics Board"></label>
<button text="%{INSTALL_LIBRARY}: PicoRobotics" callbackKey="installPyLib"></button>

# kitronik_pico_robotics__create
<block type="kitronik_pico_robotics__create">
    <value name="I2CAddress">
        <shadow type="math_number">
            <field name="NUM">108</field>
        </shadow>
    </value>
    <value name="sda">
        <shadow type="pinout">
            <field name="PIN">8</field>
        </shadow>
    </value>
    <value name="scl">
        <shadow type="pinout">
            <field name="PIN">9</field>
        </shadow>
    </value>
</block>

# kitronik_pico_robotics__adjust_servos
<block type="kitronik_pico_robotics__adjust_servos">
    <value name="change">
        <shadow type="math_number">
            <field name="NUM">0</field>
        </shadow>
    </value>
</block>

# kitronik_pico_robotics__servo_write
<block type="kitronik_pico_robotics__servo_write">
    <value name="servo">
        <shadow type="math_number">
            <field name="NUM">0</field>
        </shadow>
    </value>
    <value name="degrees">
        <shadow type="math_number">

```

```

        <field name="NUM">0</field>
    </shadow>
</value>
</block>

# kitronik_pico_robotics__servo_write_radians
<block type="kitronik_pico_robotics__servo_write_radians">
    <value name="servo">
        <shadow type="math_number">
            <field name="NUM">0</field>
        </shadow>
    </value>
    <value name="radians">
        <shadow type="math_number">
            <field name="NUM">0</field>
        </shadow>
    </value>
</block>

# kitronik_pico_robotics__motor_on
<block type="kitronik_pico_robotics__motor_on">
    <value name="motor">
        <shadow type="math_number">
            <field name="NUM">0</field>
        </shadow>
    </value>
    <value name="speed">
        <shadow type="math_number">
            <field name="NUM">0</field>
        </shadow>
    </value>
</block>

# kitronik_pico_robotics__motor_off
<block type="kitronik_pico_robotics__motor_off">
    <value name="motor">
        <shadow type="math_number">
            <field name="NUM">0</field>
        </shadow>
    </value>
</block>

# kitronik_pico_robotics__step
<block type="kitronik_pico_robotics__step">
    <value name="motor">
        <shadow type="math_number">
            <field name="NUM">0</field>
        </shadow>
    </value>
    <value name="steps">
        <shadow type="math_number">
            <field name="NUM">0</field>
        </shadow>
    </value>
    <value name="speed">
        <shadow type="math_number">
            <field name="NUM">20</field>
        </shadow>
    </value>
    <value name="holdPosition">
        <shadow type="logic_boolean">
            <field name="BOOL">FALSE</field>
        </shadow>
    </value>
</block>

# kitronik_pico_robotics__step_angle
<block type="kitronik_pico_robotics__step_angle">
    <value name="motor">
        <shadow type="math_number">
            <field name="NUM">0</field>
        </shadow>

```

```

</value>
<value name="angle">
  <shadow type="math_number">
    <field name="NUM">0</field>
  </shadow>
</value>
<value name="speed">
  <shadow type="math_number">
    <field name="NUM">20</field>
  </shadow>
</value>
<value name="holdPosition">
  <shadow type="logic_boolean">
    <field name="BOOL">FALSE</field>
  </shadow>
</value>
<value name="stepsPerRev">
  <shadow type="math_number">
    <field name="NUM">200</field>
  </shadow>
</value>
</block>

```

templates/page/blocks/definitions/sand_table_robot_dsl.md

```

# Sand Drawing Machine
<category name="Sand Drawing Machine">
<label text="Sand Drawing Machine"></label>
<button text="%{INSTALL_LIBRARY}: sandRobot" callbackKey="installPyLib"></button>
<button text="%{INSTALL_LIBRARY}: stepper" callbackKey="installPyLib"></button>

# sand_table_robot__create
<block type="sand_table_robot__create">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="motor1_pins">
    <shadow type="lists_create_with">
      <mutation items="4"></mutation>
      <value name="ADD0">
        <shadow type="pinout">
          <field name="PIN">17</field>
        </shadow>
      </value>
      <value name="ADD1">
        <shadow type="pinout">
          <field name="PIN">16</field>
        </shadow>
      </value>
      <value name="ADD2">
        <shadow type="pinout">
          <field name="PIN">15</field>
        </shadow>
      </value>
      <value name="ADD3">
        <shadow type="pinout">
          <field name="PIN">14</field>
        </shadow>
      </value>
    </shadow>
  </value>
  <value name="motor2_pins">
    <shadow type="lists_create_with">
      <mutation items="4"></mutation>
      <value name="ADD0">
        <shadow type="pinout">
          <field name="PIN">21</field>
        </shadow>
      </value>
    </shadow>
  </value>

```

```
    </shadow>
  </value>
  <value name="ADD1">
    <shadow type="pinout">
      <field name="PIN">20</field>
    </shadow>
  </value>
  <value name="ADD2">
    <shadow type="pinout">
      <field name="PIN">19</field>
    </shadow>
  </value>
  <value name="ADD3">
    <shadow type="pinout">
      <field name="PIN">18</field>
    </shadow>
  </value>
</shadow>
</value>
<value name="sensor_shoulder_pin">
  <shadow type="pinout">
    <field name="PIN">0</field>
  </shadow>
</value>
<value name="sensor_elbow_pin">
  <shadow type="pinout">
    <field name="PIN">0</field>
  </shadow>
</value>
<value name="L1">
  <shadow type="math_number">
    <field name="NUM">31.0</field>
  </shadow>
</value>
<value name="L2">
  <shadow type="math_number">
    <field name="NUM">31.0</field>
  </shadow>
</value>
<value name="steps_per_rev">
  <shadow type="math_number">
    <field name="NUM">4096.0</field>
  </shadow>
</value>
<value name="backlash_deg_m1">
  <shadow type="math_number">
    <field name="NUM">3.2</field>
  </shadow>
</value>
<value name="backlash_deg_m2">
  <shadow type="math_number">
    <field name="NUM">3.0</field>
  </shadow>
</value>
<value name="homing_dir_shoulder">
  <shadow type="math_number">
    <field name="NUM">1</field>
  </shadow>
</value>
<value name="homing_dir_elbow">
  <shadow type="math_number">
    <field name="NUM">1</field>
  </shadow>
</value>
<value name="homing_clear_steps">
  <shadow type="math_number">
    <field name="NUM">500</field>
  </shadow>
</value>
<value name="default_speed_ms">
  <shadow type="math_number">
    <field name="NUM">2</field>
  </shadow>
</value>
```

```

    </shadow>
  </value>
</block>

# sand_table_robot__off
<block type="sand_table_robot__off">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

# sand_table_robot__home
<block type="sand_table_robot__home">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
</block>

# sand_table_robot__move_line
<block type="sand_table_robot__move_line">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="target_x">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="target_y">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="segments">
    <shadow type="math_number">
      <field name="NUM">500</field>
    </shadow>
  </value>
  <value name="speed">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
</block>

# sand_table_robot__move_arc
<block type="sand_table_robot__move_arc">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="center_x">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="center_y">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="radius">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>

```

```

</value>
<value name="start_angle">
  <shadow type="math_number">
    <field name="NUM">0</field>
  </shadow>
</value>
<value name="end_angle">
  <shadow type="math_number">
    <field name="NUM">0</field>
  </shadow>
</value>
<value name="segments">
  <shadow type="math_number">
    <field name="NUM">50</field>
  </shadow>
</value>
<value name="speed">
  <shadow type="math_number">
    <field name="NUM">0</field>
  </shadow>
</value>
</block>

# sand_table_robot__draw_spiral
<block type="sand_table_robot__draw_spiral">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="max_radius">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="vindinger">
    <shadow type="math_number">
      <field name="NUM">10</field>
    </shadow>
  </value>
  <value name="segments_pr_omgang">
    <shadow type="math_number">
      <field name="NUM">60</field>
    </shadow>
  </value>
  <value name="speed">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
</block>

# sand_table_robot__draw_flower
<block type="sand_table_robot__draw_flower">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="max_radius">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="petals">
    <shadow type="math_number">
      <field name="NUM">5</field>
    </shadow>
  </value>
  <value name="speed">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>

```



```
    </shadow>
  </value>
</block>

# sand_table_robot__run_gcode_text
<block type="sand_table_robot__run_gcode_text">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="gcode_text">
    <shadow type="text">
      <field name="TEXT"></field>
    </shadow>
  </value>
  <value name="segments">
    <shadow type="math_number">
      <field name="NUM">40</field>
    </shadow>
  </value>
  <value name="draw_speed_ms">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="travel_speed_ms">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="reset_modal">
    <shadow type="logic_boolean">
      <field name="BOOL">TRUE</field>
    </shadow>
  </value>
</block>

# sand_table_robot__run_gcode_file
<block type="sand_table_robot__run_gcode_file">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="path">
    <shadow type="text">
      <field name="TEXT"></field>
    </shadow>
  </value>
  <value name="segments">
    <shadow type="math_number">
      <field name="NUM">40</field>
    </shadow>
  </value>
  <value name="draw_speed_ms">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="travel_speed_ms">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="reset_modal">
    <shadow type="logic_boolean">
      <field name="BOOL">TRUE</field>
    </shadow>
  </value>
</block>
```

```
# ST7735S
<category name="ST7735S">
<label text="ST7735S"></label>
<button text="%{INSTALL_LIBRARY}: st7735s" callbackKey="installPyLib"></button>

# color565
<block type="color565">
  <value name="r">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="g">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="b">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
</block>

# st7735_s__create
<block type="st7735_s__create">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="dc_pin">
    <shadow type="pinout">
      <field name="PIN">0</field>
    </shadow>
  </value>
  <value name="rst_pin">
    <shadow type="pinout">
      <field name="PIN">0</field>
    </shadow>
  </value>
  <value name="cs_pin">
    <shadow type="pinout">
      <field name="PIN">0</field>
    </shadow>
  </value>
  <value name="width">
    <shadow type="math_number">
      <field name="NUM">160</field>
    </shadow>
  </value>
  <value name="height">
    <shadow type="math_number">
      <field name="NUM">128</field>
    </shadow>
  </value>
</block>

# st7735_s__fill
<block type="st7735_s__fill">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="color">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
```

</block>

st7735_s__fill_rect

<block type="st7735_s__fill_rect">

<value name="id">

<shadow type="math_number">

<field name="NUM">1</field>

</shadow>

</value>

<value name="x">

<shadow type="math_number">

<field name="NUM">0</field>

</shadow>

</value>

<value name="y">

<shadow type="math_number">

<field name="NUM">0</field>

</shadow>

</value>

<value name="w">

<shadow type="math_number">

<field name="NUM">0</field>

</shadow>

</value>

<value name="h">

<shadow type="math_number">

<field name="NUM">0</field>

</shadow>

</value>

<value name="color">

<shadow type="math_number">

<field name="NUM">0</field>

</shadow>

</value>

</block>

st7735_s__pixel

<block type="st7735_s__pixel">

<value name="id">

<shadow type="math_number">

<field name="NUM">1</field>

</shadow>

</value>

<value name="x">

<shadow type="math_number">

<field name="NUM">0</field>

</shadow>

</value>

<value name="y">

<shadow type="math_number">

<field name="NUM">0</field>

</shadow>

</value>

<value name="color">

<shadow type="math_number">

<field name="NUM">0</field>

</shadow>

</value>

</block>

st7735_s__hline

<block type="st7735_s__hline">

<value name="id">

<shadow type="math_number">

<field name="NUM">1</field>

</shadow>

</value>

<value name="x">

<shadow type="math_number">

<field name="NUM">0</field>

</shadow>

</value>

```

<value name="y">
  <shadow type="math_number">
    <field name="NUM">0</field>
  </shadow>
</value>
<value name="w">
  <shadow type="math_number">
    <field name="NUM">0</field>
  </shadow>
</value>
<value name="color">
  <shadow type="math_number">
    <field name="NUM">0</field>
  </shadow>
</value>
</block>

```

```

# st7735_s__vline
<block type="st7735_s__vline">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="x">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="y">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="h">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="color">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
</block>

```

```

# st7735_s__rect
<block type="st7735_s__rect">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="x">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="y">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="w">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="h">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>

```

```
</value>
<value name="color">
  <shadow type="math_number">
    <field name="NUM">0</field>
  </shadow>
</value>
</block>
```

```
# st7735_s__line
<block type="st7735_s__line">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="x0">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="y0">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="x1">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="y1">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="color">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
</block>
```

```
# st7735_s__text
<block type="st7735_s__text">
  <value name="id">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
  <value name="string">
    <shadow type="text">
      <field name="TEXT"></field>
    </shadow>
  </value>
  <value name="x">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="y">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="color">
    <shadow type="math_number">
      <field name="NUM">0</field>
    </shadow>
  </value>
  <value name="size">
    <shadow type="math_number">
      <field name="NUM">1</field>
    </shadow>
  </value>
```

```
</shadow>  
</value>  
</block>
```